GigBid

Research & Development Report - Interim Draft

Individual AI-4016 Assignment II | Development checkpoint: 7 November 2026

Status	Evidence at this checkpoint
Prototype	A local React/FastAPI application, LangGraph bid workflow, Portfolio MCP evidence server, and synthetic Marketplace MCP adapter exist.
Verified	Five automated tests and the React production build pass. The Docker stack runs; a pending approval survived an API restart and was then approved.
Unavailable	The instructor's GigSim starter kit and official evaluation data were not attached to the Classroom assignment at the time of this report.
Submission	This is an interim draft. It must not be treated as the final measured R&D; submission.

Research question

Can an evidence-grounded, human-reviewed agent workflow prepare safer freelance proposals while preserving a traceable decision and preventing unapproved marketplace writes? The current prototype tests the engineering path. It cannot answer marketplace quality or performance questions until the official simulator, personas, and evaluation split are available.

1. System design

The web inbox sends a job identifier to FastAPI. A typed marketplace port loads the job through the synthetic Marketplace MCP server in the Docker stack. A LangGraph fan-out runs an analyst, risk assessor, and fit matcher in parallel. The fit matcher calls the Portfolio MCP server's **search_evidence** tool through a real MCP stdio session. A decision node selects bid, watch, or skip from configuration thresholds. The bid branch prepares a price and a cited proposal draft, checks claim support and prohibited phrases, and pauses at a human approval interrupt. A synthetic draft is created only after approval of the unchanged text.

Boundary	Current implementation	Remaining integration
Marketplace	Typed port with synthetic MCP-backed draft adapter	Instructor GigSim adapter and durable write idempotency
Evidence	Portfolio MCP server over synthetic records	Consented CV, repositories, case studies, and evaluated retrieval
State	Memory locally; optional Postgres checkpointer in Docker	Restart test, outbox, scheduling, multi-tenant isolation
Human control	Dry-run preview and approval interrupt	Full reviewer studio and conversation escalation

Safety invariants

The demo adapter performs no live marketplace action. Rejection creates no draft. An edited draft is held for renewed claim and compliance review. The approval path uses a separate idempotency key for the real draft. The API can require reviewer HTTP Basic authentication when both environment variables are configured; public deployment must use HTTPS and stronger tenant controls.

Traceability

Each graph run stores state and event labels, including extracted requirements, risk check, portfolio fit, decision, price, draft, compliance, and human action. A containerized run paused at approval, survived an API restart through Postgres, and finished after approval. This demonstrates checkpoint recovery for one synthetic run, not exactly-once marketplace writes under arbitrary failure.

2. Verification and R&D; method

The present checks establish only that the local development slice behaves as designed. They are not measurements on the assignment's hidden or official data.

Check	Observed result	Interpretation
Automated Python tests	5 passed on 7 Nov 2026	Approval and rejection paths, MCP adapter behavior, and reviewer authentication work in the local test process.
React production build	Passed on 7 Nov 2026	TypeScript and Vite compile the current UI.
MCP protocol call	search_evidence returned typed synthetic records	The portfolio server can be called over MCP, and the fit node uses that path.
Docker Compose	Stack started and HTTP route responded	An MCP-backed pending approval survived API restart; approval created one synthetic draft. A risk-flagged job skipped without a draft.

Planned evaluation after the official kit arrives

Preserve the instructor's provided fixtures and train/dev/test split. Register the exact corpus and parser versions. Compare a transparent keyword baseline against at least four retrieval variants, including sparse, dense, hybrid, and reranked retrieval where feasible. Measure retrieval recall at k, citation support, unsupported claim rate, proposal quality with calibrated human judging, latency, and cost. Report confidence intervals and results by persona and job type. Include an ablation for evidence retrieval and a baseline graph comparison. Use the prescribed GigSim outcomes and release no hidden labels into the model prompt.

Release gates

The final report should include the exact dataset count, exclusions, label rubric, annotator agreement, seeds, model and prompt versions, per-variant tables, failure examples, trace links, and reproducible commands. None of those values is estimated in this interim draft.

3. Limitations and next steps

The synthetic jobs and evidence are small and hand-written. The deterministic writer does not demonstrate LLM reasoning or response quality. The current graph lacks the required conversation workflow, workspace MCP layer, real model service, scheduled worker, full RAG study, browser end-to-end suite, public deployment, and portfolio publication. The only verified outcome is prototype behavior on the included examples.

Priority	Required work	Acceptance evidence
1	Obtain and inspect the official GigSim kit and evaluation data	Adapter contract, unchanged fixtures, data card
2	Complete marketplace and workspace MCP access, conversation graph, and persistent writes	Inspector snapshots, restart and isolation tests
3	Run measured RAG and graph evaluations	Reproducible scripts, labeled results, human panel, ablations
4	Deploy, publish, record, and submit	Live URLs, final commit, three-minute video, final PDF, private reviewer comment

Integrity note

No result in this document is described as an instructor evaluation. The report should be revised after the complete system and official data are available. A final submission must provide working public links and one coherent Classroom submission before the deadline.

Source and reproducibility

Implementation source: local GigBid repository, main branch, development checkpoint. Assignment requirements: AI-4016 Assignment II brief in the course Classroom item, accessed 7 November 2026. Commands: **PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 .venv/bin/python3 -m pytest -q**, **npm run build**, and **docker compose -f infra/docker-compose.yml config --quiet**.